

Collaborative tools and processes to support software engineering shift work

I Gorton, I Hawryszkiewicz and K Ragoonaden

This paper describes the construction of and early experiences with a software engineering support environment for projects using globally distributed teams. The goals of the project are twofold. Firstly, it aims to construct a pragmatic solution to the problems experienced by widely geographically dispersed groups which collaborate on software development projects. Secondly, it aims to experiment with processes which facilitate software shift work through exploitation of time differences between collaborating groups. The construction of the support environment, known as GWSE (Global Working in Software Engineering) system, is presented, including its architecture and integration with existing workflow, document and project management tools. The use of the GWSE system in a trial development project is then described, including an initial quantitative analysis of the collaboration overheads experienced.

1. Introduction

Software engineering is an inherently collaborative, team-based process [1]. In all phases of a software project, software engineers are required to collaborate, communicate and co-ordinate. In contrast, however, the vast majority of existing software engineering tools, ranging from program editors to powerful CASE tools, are designed to support only single-user activity [1]. Such tools hamper collaboration and information sharing, making the potential benefits of groups of developers working closely together difficult to realise [2].

To address this issue, several authors have advocated the application of computer-supported collaborative work (CSCW) or groupware technology [3, 4] to a range of software engineering activities [1, 5, 6]. Groupware technology provides pragmatic support for both synchronous and asynchronous collaboration on software development tasks, and is applicable to both co-located and geographically distributed software development teams.

In the GWSE (Global Working in Software Engineering) project [7, 8], groupware technology is being used to develop tools to support the co-ordinated working and process management of widely distributed groups of software developers. The aims of the project are:

- to facilitate co-ordination and communications among geographically separated engineers,
- to provide project planning, management and monitoring tools for distributed development teams,

- to enable efficient and safe sharing and modification of process and development artefacts between groups,
- to provide automatic process metrics gathering and processing tools.

In addition, the GWSE tool-set is being used to explore the potential for using time-displaced developers to achieve the effect of software shift work. Using globally distributed teams for software projects creates the opportunity for performing 24-hour software development [9]. Time differences between teams could be gainfully exploited to give the effect of continuous and concurrent work on the project [10, 11], potentially shortening the development cycle and reducing significantly the time-to-market for products [12]. The term 'overnight gain effect' [13] has been defined to describe this phenomenon.

The remainder of this paper describes the project context, requirements and a supporting architecture for the GWSE system. A brief example of using the system is then presented, which illustrates some its features. Finally the results of a case study in time-displaced working using the GWSE system are discussed.

2. System context and requirements

The basic goals of the GWSE system are to provide:

- software process description tools,
- project management and monitoring facilities,

- workflow management for defined processes,
- transparent document sharing, archiving and management,

to both co-located and widely distributed software development teams. Hence the system must be able to provide distributed co-ordination, communication and document management between groups of developers at several sites.

These requirements are constrained by one important factor. In order to build a working system in minimum time, and to increase the attractiveness for projects to adopt the system, existing tools need to be used wherever possible. This dictates a philosophy of integrating widely used project, document and workflow management tools and products to construct a fully functional system.

The importance of providing a tool-set that is attractive to developers and amenable to adoption by project teams cannot be over-emphasised. This is due to the dual, but complementary foci of the project. The objectives are not only to build a useful set of tools for distributed development groups, but also to investigate different processes in distributed teams which can provide significant overnight gain at minimum risk. In this respect, reductions in the development cycle are only likely to be experienced if the collaboration and communications overheads associated with distributed working are low. Studies of co-located development teams in large projects have shown that up to 70% of developer's time is spent collaborating [14], and 85% of project costs can be accounted for by team activities [15]. Given the additional difficulties of asynchronous collaboration, it would seem possible that collaboration overheads could easily increase to completely negate any associated overnight gains.

At a project planning level, some criteria for projects that are amenable to distributed teams have been identified [11]. However, to advantageously exploit time differences, experimentation with much finer-grain activities at appropriate stages of the development process is required. Initially, it was planned to concentrate on distributing and overlapping detailed design, coding and testing processes. Intuitively, these tasks appeared to require less direct communications than activities earlier in the development life cycle. This belief is supported by the findings of a study by Kraut and Streeter [16]. In their extensive study of 563 people and 65 projects at the software development division of an R&D company, they found that formal, impersonal processes are more widely used by larger projects which had passed their requirements and design stages.

Given this context, the following approaches have been adopted:

- simple, graphical process modelling language — the processes to be used on a project should be defined in a notation familiar to project managers, and any tools created should interoperate with existing project management tools,
- distributed architecture — each development site would maintain its own copy of the workflow management system which guides the process enactment,
- top-down approach — the project plan would define a common, albeit co-operative, process which is followed by all sites participating in the project.

3. GWSE system architecture

The GWSE system architecture is based around a distributed collection of workflow management and document servers (see Fig 1). It is anticipated that the most common scenario will see each development site in the project maintain its own GWSE servers for use by the local developers (although this is not a prerequisite — different groups may share servers at one site).

The architecture aims to support co-located developers by attempting to ensure that the majority of accesses to information in the workflow system and document archive are local. This means collaboration, co-ordination and document access will be carried out over the relatively fast local area network. Access to remote data is therefore minimised, but, when needed, will usually be slower due to the current performance characteristics of wide area networks.

The architecture comprises two distinct databases, the workflow database and the document archive database. Each database needs to be set up and maintained for each distinct project which uses the GWSE system.

While both databases are distributed, the mechanisms employed for maintaining distribution are different. The workflow database, which contains process-related documents such as task assignments, status reports and review reports, is fully replicated at each site. All accesses to workflow documents are consequently local, with periodic document updating between replicas to keep the copies of documents consistent. The document database, which contains product-related documents such as design and test specification, and program code, is, however, partitioned across sites. This means only one copy of each document exists across all partitions. Consequently, an access to a document by a developer at one site may be satisfied locally, or may need to access a remote document server at another site where the document archive is stored.

The different distribution strategies for the two databases are justified as follows. The workflow database

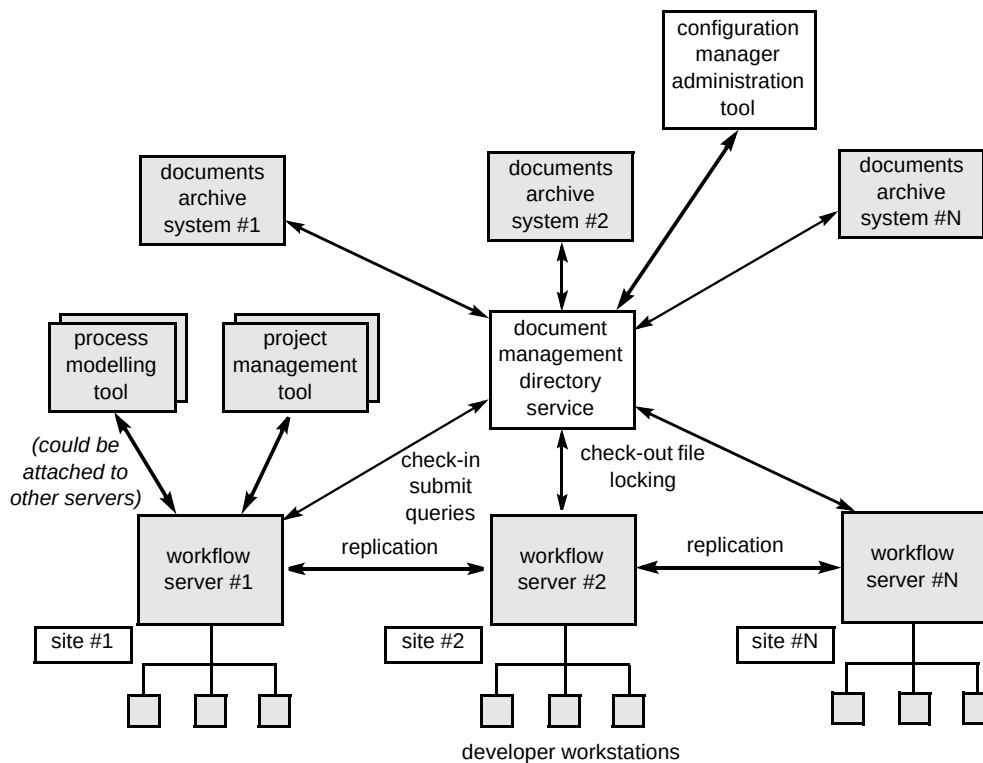


Fig 1 GWSE system architecture.

contains many small documents containing information on the tasks in the project, including their status and dependencies, and the person or role assigned to the task. When a document is updated, it can be quickly replicated to other sites, even across slow links, due to its small size. Also, most workflow documents may only be modified by one person (the task owner), thus greatly reducing the potential for inconsistency when conflicting modifications are made simultaneously at different sites.

Of course, any replication mechanism will, however, incur a slight delay from when a document is saved until its changes are visible at all sites. It is therefore possible that a developer may open and view a task document assigned to someone else, and not see the most current version. This may very occasionally cause problems if it is critical that up-to-date information is available. However, in such circumstances, options to force immediate replication of the local database can be used to minimise replication delays.

Unlike the workflow database, the characteristics of the document database do not lend themselves so readily to full replication, for two main reasons. Firstly, these project-related documents are frequently very large — system specifications, designs and executables can easily be of the order of many megabytes in size. For such documents, replication would have much greater implications on network traffic, and reliability of transfer would be an issue. This is especially true if slow, remote dial-up links or the Internet were used to connect sites.

Secondly, project documents such as designs and code can usually be modified by several team members. This makes it necessary to lock documents to prevent conflicting modifications being made simultaneously by different people. (Merging tools can help here, but many such conflicts are still mostly undesirable). In a fully replicated environment, remote locking mechanisms would be needed to lock all copies of a document. These are problematic, however, in situations when a file is simultaneously opened at two sites, or when networks or servers fail.

While these problems are by no means insurmountable [17], they introduce unnecessary complexities, which are not present when a database partitioning strategy is adopted. Documents then only need to be transferred to remote sites when a team member actually wants to view or update the file — this reduces network load. Any lock or multi-lock placed on a document is immediately visible to all team members, effectively preventing conflicting modifications being made¹.

In order to meet the requirement of location-transparent access to documents in the document database, an additional component, the document management directory service, has been incorporated into the architecture. Whenever a new document is added to the database, its location (server and path) is stored in the directory database.

¹ The downside of partitioning is that a server or network failure can make a document unavailable at a remote site, introducing delays in the development process. Replication is more resilient in the face of equipment failure, but even this does not solve all problems. In any case, such circumstances are considered rare.

Subsequently, the directory can be used to locate any document in the project. In the rare case of document name clashes, the directory returns the locations of all matching documents, allowing the user to select from the list until the correct file is located.

The final component of the GWSE architecture is the project management (PM) sub-system. This comprises two distinct tools, one for process modelling and one for project management and monitoring. The process modelling tool provides a simple graphical notation based on PERT charts for defining the desired processes, deliverables and task dependencies. Once completed the process model is used as the basis for detailed project planning using Gantt charts within standard project management tools.

From the project schedule, the PM sub-system is responsible for automatically generating the initial task documents and required workflows in the workflow database. Once the workflow documents have been generated by the PM sub-system, the data replication facilities are used to distribute the project set-up to all participating sites. This has the desirable effect of making the location of the project manager and the project plan totally transparent. The PM tools can interface to any workflow server to download new tasks, or update the plan from the workflow documents to reflect project status. Such location-independent project management is seen as especially important in dynamic, widely distributed, virtual team environments.

In addition to catering for distributed project planning, the process modelling tool supports collaborative editing of a process model. Project managers and developers from different sites can view and edit the same process model simultaneously, in a WYSIWIS (What You See Is What I

See) fashion. The tool ensures consistency of the model by enforcing a simple collaboration protocol among concurrent users.

4. GWSE system implementation

In order to construct a working system in minimum time, several commercial-off-the-shelf (COTS) components have been integrated, and supplemented with additional functionality to form the GWSE system. Figure 2 illustrates the physical architecture of the system.

The core of the system architecture, the workflow management engine, is based on Lotus Notes. Notes provides industrial-strength replication and distribution services, as well as useful features to implement workflows and event triggers, and other asynchronous groupware features.

The document archive database has initially been built upon the features of Intersolv's PVCS version control system, which provides source code and document archiving. The implementation, however, is designed to support a generic document database service, facilitating the replacement of PVCS with any appropriate system with a programmatic interface.

The process modelling tool, SEP-Tool, has been implemented in C++ using CORBA as a distribution and synchronisation technology. The modelling tool interfaces with the project management tools and capabilities which are provided by Microsoft Project. MS Project has been supplemented with a set of new options which generate the Notes database from a Gantt Chart, and update the Gantt Chart from the information in the workflow database as the project proceeds.

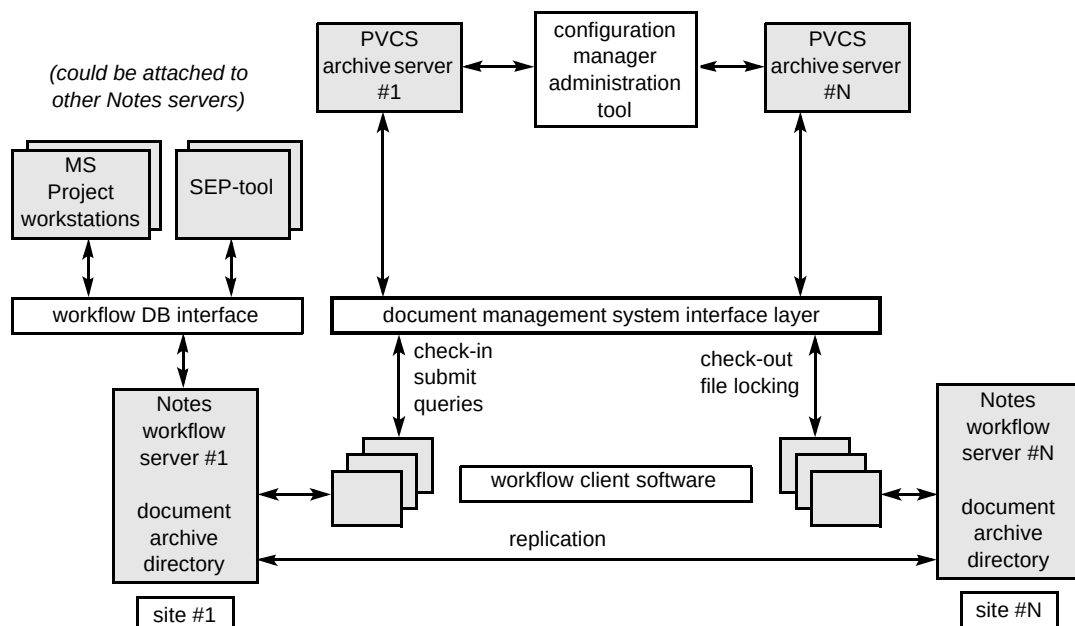


Fig 2 GWSE system physical architecture.

Team members interface to the workflow and document database servers through a native Microsoft Windows client interface, written in Visual Basic and C++. Notes macros periodically scan the workflow database, and trigger the necessary actions upon state changes in the task documents. The definition of users and role management is handled by a Notes database, along with the document database directory services, which is actually replicated across the workflow servers.

5. GWSE system trials

The GWSE system was first used during a live development project comprising four software engineers, situated at two different sites in Sydney. In addition to completing the designated project, the main interest was in exploring the overheads involved in using the GWSE system in a distributed, time-displaced development environment, and in measuring the extent to which the overnight gain effect could be exploited. A previous case study [7] in distributed team development using a prototype version of the GWSE system had produced preliminary figures of collaboration overheads (22% of project time, including 17% asynchronous and 5% synchronous) and overnight gain effect exploitation (82% of all opportunities).

To achieve these aims, all four developers were assigned to work a maximum of 50% of their time on this project over an estimated duration of three weeks. Two collocated team members were assigned to work on the project during the mornings (approximately 8am — 1pm), and the other two co-located team members were assigned to work on the project during afternoons (approximately noon to 5pm).

The process defined for the project essentially followed a waterfall life cycle model. In addition, the two groups were designated to co-operate according to the rules of the delegation model of co-operative software engineering [13]. In the delegation model, one group is responsible for designing and developing code modules. Completed modules are then passed to the remote testing group which performs tests from the test specifications they have developed, and reports the failures observed to the development group. The development group consequently fixes problems and instructs the test group to re-test. Responsibility for overall task completion and monitoring remains with the supervisory development group.

To exploit the overnight gain effect, the development group can allocate testing tasks to the test group at the end of their working day. The test group runs as many tests as possible during their day, and hands back the results at the end of their working day to the development group. In this manner, from the development group's perspective, testing has occurred overnight. Also, code and document reviews

can be allocated to remote group members so that these can be completed 'overnight'.

In the trial project, one pair of collocated engineers were assigned all design and development tasks, and the other pair were assigned all test specification and testing responsibilities. As well, all synchronous communication between groups was prohibited except for the one hour period from noon to 1pm, which could be used legitimately to discuss issues on the telephone. Outside this period, all communication between groups was restricted to the GWSE system or e-mail.

This arrangement effectively simulated two time-displaced small groups with only one hour of overlapping work time. In this manner, it was hoped to observe productivity gains through exploitation of the overnight gain effect during testing and review of deliverables. To this end, all team members filled in detailed daily time sheets to record their activities on the project. They also recorded when tasks were allocated to the remote group for completion 'overnight', and whether the tasks were successfully completed or not.

6. Planning the trial project

The GWSE system was set up to operate over the two sites, with a workflow and document server at each site. The project plan was developed using the GWSE extensions to MS Project, as illustrated in Fig 3. The completed plan and schedule was then used to generate the corresponding tasks in the workflow system ready for process enactment.

In order to maximise opportunities for exploiting the overnight gain effect and hence complete the project in a shorter elapsed time, two steps were taken.

- Short duration tasks

The project schedule was decomposed into as many short duration (approximately 0.5-3 days), potentially concurrent tasks as could be reasonably anticipated. Short-duration tasks provide more frequent opportunities to utilise remote group resources, and hence reduce task time-to-completion.

- Collaborative tasks

A number of tasks, including unit, system and integration testing, were allocated to collaborating engineers from different sites. These arrangements, although potentially increasing collaboration overheads, created opportunities for work to be completed 'overnight'.

Further, to help with task collaboration, a task hand-over time was designated. A handover time defines the time

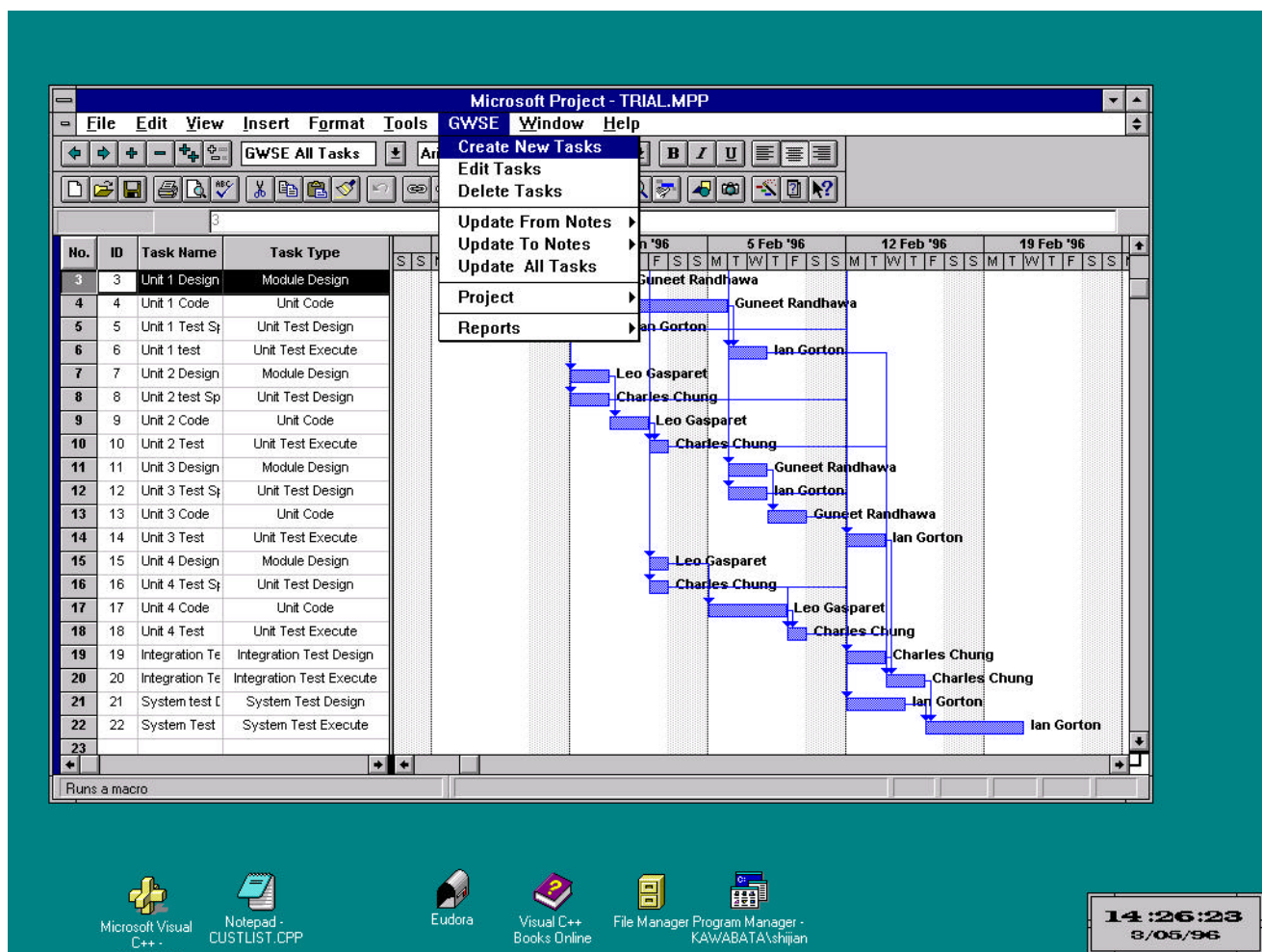


Fig 3 The trial project plan.

by which a team member should submit 'status reports' on all collaborative tasks, and ensure that any deliverables are made available for other team members to use. In this project, the handover time was specified to be during the overlap period so that remedial actions could be immediately commenced if task handovers did not take place when planned.

7. Development trial — proceedings

The project progressed very much as planned, with developers from different sites collaborating asynchronously in order to design, develop and test the software. Throughout the trial, team members used various views of the GWSE system to monitor the progress of tasks. For example, when a team member entered the GWSE system, they initially saw the list of 'ready' or 'in progress' tasks that have been assigned to them (see Fig 4). This screen is automatically updated as new tasks are assigned to the team member, and as tasks are completed.

information on the task status, immediate predecessor and successor tasks, review and approval mechanisms, and data on the actual start and end dates and the amount of effort needed to complete. From any task form, the user can navigate easily to related tasks, and view and access the deliverables of the task which have been stored in the document management system.

Upon completion of a coding task, or code unit integration task for sub-system or system testing, the associated testing tasks were automatically placed in the 'ready' state. Sequences of tests were then performed based on the appropriate test specification, and any test case which produced incorrect results was reported by creating a fault report (FR) form. FRs record the test case that failed, the symptoms of the failure, and the project error category to allow classification of error types. Upon creation, the FR automatically appears in the to-do list of the programmer who created the code, or in the case of integration or system testing, the team member assigned to manage that particular testing phase. At all times, each FR is the responsibility of a given developer, and a public view allows the status of FRs to be monitored (see Fig 6).

Task Id	Rev	Description	Owner	Status	% Comp	Finishing
2		Requirements	Ian Gorton	Review in progress	100%	26/01/96
14		Unit 3 Test	Ian Gorton	In progress	100%	10/02/96
12	1	Unit 3 Test Spec	Ian Gorton	Approval in progress	100%	14/02/96
		Review for Task 16	Ian Gorton	Ready		01/02/96
5		Unit 1 Test Spec	Ian Gorton	In progress	50%	31/01/96

Fig 4 Private to-do list.

Fig 5 An example task form.

Id	Rev	Task Description	Task Owner	Status	% Comp	Finishing
6		Unit 1 test	Ian Gorton	Complete	100%	08/02/96
		Fault Report No. 1 for Task 6	Status: Complete			
		Fault Report No. 2 for Task 6	Status: Complete			
		Fault Report No. 3 for Task 6	Status: Complete			
		Fault Report No. 4 for Task 6	Status: Complete			
		Fault Report No. 5 for Task 6	Status: Complete			
		Fault Report No. 6 for Task 6	Status: Complete			
		Fault Report No. 7 for Task 6	Status: Complete			
10		Unit 2 Test	Charles Chung	Complete	100%	03/02/96
		Fault Report No. 1 for Task 10	Status: Complete			
		Fault Report No. 2 for Task 10	Status: Complete			
14		Unit 3 Test	Ian Gorton	Complete	100%	11/02/96
		Fault Report No. 1 for Task 14	Status: Complete			
		Fault Report No. 2 for Task 14	Status: Complete			
		Fault Report No. 3 for Task 14	Status: Complete			
		Fault Report No. 4 for Task 14	Status: Complete			
		Fault Report No. 5 for Task 14	Status: Complete			

Fig 6 Tracking testing tasks.

The project was completed just ahead of schedule, with very little synchronous communications between developers. Only in one instance did a sever problem force two remote team members to solve a problem on the telephone — the problem arose in overlap time and its resolution occurred after the overlap time had ended.

8. Analysis

Upon project completion, the data from each engineer's time sheet was collated. Figure 7 shows the breakdown of time spent on each activity during the project

A total of 66% (11% design + 21% coding + 12% test + 14% test spec + 8% review) of the recorded hours were

spent on design, coding, testing and review activities. Importantly, only 19% of recorded hours represented use of the GWSE system, and 3% of the project was consumed by synchronous interaction. Consequently, 22% of the project time accounted for all group co-ordination, configuration management and project management and monitoring.

Although research in asynchronous distributed interaction remains scarce [18], this figure of 22% corresponds very closely with the findings of an earlier case study [7]. It would, however, seem too early to begin to draw any strong conclusions at this stage, as although both case studies employed very similar processes, there were many variable factors such as GWSE system and project complexity, and different personnel. However, the results would seem to indicate that, in this instance, the overhead introduced by distributed working when using CSCW tools was not overly significant or prohibitive. In fact, probably due to the process and tools used, the amount of collaboration time is somewhat less than in other, albeit much larger, studies such as Jones [14] and DeMarco and Lister [15].

In terms of overnight gain effect exploitation, Fig 8 shows the results of the collected data. Each team member recorded each day if they were expecting outstanding units of work to be completed 'overnight'. If so, they recorded whether all, some or none of the deliverables were completed when expected.

Interestingly, in one third of all cases, team members did not expect any work packages to be completed asynchronously by a collaborating team member. This occurred when a task, such as a module design or coding, extended over a number of days, which was the case for a number of tasks even in this small project. In larger projects with tasks of longer duration, it is reasonable to expect this figure to be much larger.

Importantly, however, all expected deliverables were completed overnight in approximately 88% of the remaining cases, and only in 9% of cases did team members fail completely to synchronise activities. Failure to synchronise cost the project an estimated 8 hours in stalled tasks, which accounts for part of the 'waiting' total in Fig 7.

These measures give a good indication that distributed asynchronous working in software engineering, when supported by groupware technology, is a feasible and potentially attractive proposition. The results presented above concur closely with those from a previous case study. Together they give increased confidence in the suitability of global working and the GWSE system for small teams usage.

	notes/ Email	code	test	synchronous	test spec	review	design spec	waiting	engineer totals
tester #1	6.25	0	12.75	2	14.75	4.25	11	3.75	54.75
tester #2	5.5	0	7.5	2.25	10	8	0	5	38.25
programmer #1	11.5	23.5	0	0.5	0	1.5	3.75	6.5	47.25
programmer #2	9.75	14	0	0.25	0	0.75	4.25	6.5	35.5
activity totals	33	37.5	20.25	5	24.75	14.5	19	21.75	175.75
percentages	19%	21%	12%	3%	14%	8%	11%	12%	

Fig 7 Project timesheet totals.

Total (partial) days worked:	55 days
Overnight deliverables complete:	33 (60%)
Some overnight deliverables complete:	1 (2%)
No overnight deliverables complete:	3 (5%)
No overnight deliverables expected:	18 (33%)
Estimates of hours worked delayed due to overnight deliverable incompleteness: 8 hours	
Type of tasks completed overnight:	
Code test and fault fixing:	15
Task review completion:	16
Task approvals:	8

Fig 8 Overnight gain effect exploitation.

9. Current status

The GWSE system currently supports the following major features:

- project plan definition using MS Project facilities,
- automatic generation of project tasks in the workflow database from the project plan,
- controlled process enactment through a workflow management engine,
- dynamic update of the project plan from the current status of the tasks in the workflow database as the project proceeds,
- a personal information manager (PIM) tool for individual developers to facilitate tracking of tasks assigned to them — a ‘to-do’ list,
- support for automatic distribution of documents for the processes of inspection, review and approval of project deliverables,
- support for archiving of all project deliverables, and provision of transparent access to archived documents regardless of the exact location of their storage,
- support for project role definition for team members, allowing task allocations to be based on roles, and providing security to documents based on role,
- provision of multiple views of the project status for both project management and team members,

- support for a rigorous, archived, four-stage change management process (task revision, change review, approval and notification to dependent tasks),
- report generation tools to produce process and product metrics on measures such as defect detection rates and effort expended on tasks and the project.

Work is currently under way to complete the SEP-TOOL process modelling environment. We are also working on the integration of other document and configuration management systems, in addition to PVCS, with the workflow engine.

10. Comparison with related work

The link between CSCW systems and software engineering environments has been recognised by others [1, 5, 6, 19]. Specifically, a number of process-centred software engineering tools, or IPSEs, have been developed [20, 21]. Of these, the Oz system explicitly aims to support geographically distributed software development teams [21]. Specifically, Oz supports:

- decentralised (as opposed to distributed) development, enabling different process models at different sites to interoperate, with no replication of artefacts,
- a specialised, rule-based process modelling language,
- novel concurrency control mechanisms for sharing development artefacts [22],
- ‘bottom-up’ construction of multi-site environments, as opposed to ‘top-down’ decomposition of a single process across all collaborating sites.

In contrast, the approach taken by the GWSE project is ‘top-down’ and distributed, allowing a single process definition to cater for all sites through replication. Also, the GWSE system has explicitly taken a pragmatic approach to process modelling and document sharing. This has been achieved by leveraging and adapting existing project and configuration management tools and techniques. It is hoped that, in this way, actual use of the technology can more readily be encouraged by providing familiar tools and concepts for teams to adopt.

The potential for employing concurrent engineering techniques in software development has also been recognised previously, most notably by Aoyama [11]. However, this work focused on introducing concurrency at a coarse-grain of task definition. While the intention has been to leverage these ideas in the GWSE project, in addition the achievement of effective concurrent activities with fine-grain tasks through exploiting time differences has been looked at. This is believed to be a novel approach, and hence investigating the implications of this mode of work in real projects has become the primary focus of this work.

11. Further work

Current work is being concentrated in the following two main areas:

- completion of the GWSE environment, especially the groupware SEP-TOOL for process modelling,
- deployment of the GWSE system with live projects.

The key area of interest is in performing several further controlled case studies (such as in Kitchenham et al [23]) to more fully assess the effects of distributed, time-displaced development efforts. However, performing such case-studies would seem to be a particularly demanding research problem. This seems to be for a range of reasons, from simply pragmatic (e.g. GWSE system installation and training at distant remote sites) to socio-technical (finding supportive management, appropriate projects, and personnel well suited for collaborative work). Nevertheless, such case studies are needed to thoroughly investigate the advantages and disadvantages of deploying the processes and tools developed.

There is also a need to devise metrics which clearly show the degree of exploitation of the overnight gain effect, and relate this directly to the expected reduction in project schedule. Like Aoyama [11], experience argues that managing highly concurrent, distributed processes incurs overheads for which there is a need to explicitly plan and budget. Consequently, through further case studies it is hoped that a predictive model can be found and validated which will help greatly in performing cost-benefit studies and estimates for projects that are candidates for distributed developments.

12. Conclusions

This work has been motivated by the needs of global organisations to exploit their geographically distributed software development workforce in a flexible and efficient manner. Given well-designed support tools and processes, teams comprising appropriately skilled developers from all parts of an organisation could be rapidly formed. This would alleviate the need to support duplicate skill sets at each site, and facilitate lower development costs through the use of skilled work forces in countries with lower overheads.

In addition, groups in different time zones may collaborate to achieve software shift work in an attempt to reduce the development cycle for a project. Short duration tasks and asynchronous collaborative processes can be used to exploit the overnight gain effect.

Through a series of progressively complex prototypes and trials, the GWSE project has developed a system to support geographically distributed teams. The system and a set of novel, collaborative processes have been trialled, with encouraging results. There is now a level of confidence that the system architecture and implementation is sufficiently flexible, scalable and resilient for use in demanding geographically distributed project environments. Current plans are to deploy the system in both centralised and distributed project teams during 1996. These initial deployments will be closely monitored. Throughout, the aim is to continue to measure the overheads and productivity gains associated with distributed asynchronous collaboration and global working on software projects.

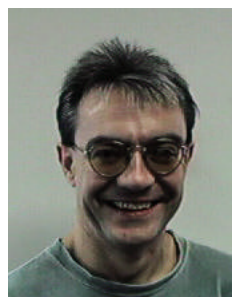
Acknowledgements

The authors would like to acknowledge the efforts of the following members of the project team over the last 3 years — Sanjeev Motwani, Lawrence Fung, Dave Beale, Charles Chung, Shijan Lu, Leo Gasparet, Guneet Randhawa and Krishnan Venkatesan. They would also like to take this opportunity to thank those that have supported this project at BT Laboratories, especially the project representative, Subhash Modasia; and last, but not least, all the support that has been provided by BT Australasia and the Australian Research Council has been much appreciated.

References

- 1 Vessey I and Sravanapudi A P: 'CASE tools as collaborative support technologies', *Comms ACM*, **38**, No 1, pp 83—95 (January 1995).
- 2 Sellars P: 'IPSEs in support of teams, in automating systems development', (Eds: Benyon and Skidmore) Plenum Press, New York (1987).
- 3 Grudin J: 'CSCW — history and focus', *IEEE Computer*, **27** No 5, pp 19—26 (May 1994).

- 4 Reinhard W, Schweitzer J, Voelksen G and Weber M: 'CSCW tools: concepts and architectures', *IEEE Computer*, 27, No 5, pp 28—36 (May 1994).
- 5 Ben-Shaul I Z and Kaiser G E: 'Integrating groupware activities into workflow management systems', in 7th Israeli Conference on Computer Based Systems and Software Engineering, Tel Aviv, Israel, pp 140—149 (June 1996).
- 6 Wastell and White P: 'Using process technology to support cooperative work: prospects and design issues', in Diaper and Sanger (Eds): 'CSCW in Practice: an Introduction and Case Studies', Springer-Verlag (1993).
- 7 Gorton I, Hawryszkiewicz I and Fung L: 'Enabling software shift work with groupware: A case study', in the Proceedings of the 29th Hawaii International Conference on System Sciences, 3, Informations Systems Track, pp 72—81 (January 1996).
- 8 Hawryszkiewicz I and Gorton I: 'Workflow support for change management and concurrency', in the Proceedings of Software Engineering: Education and Practice Conference, Otago, New Zealand, pp 181—188 (January 1996).
- 9 Rafii F and Perkins S: 'Internationalising software with concurrent engineering', *IEEE Software*, 12, No 5, pp 39—46 (September 1995).
- 10 Prasad B, Morenc R S, Rangan R M: 'Information management for concurrent engineering — research issues', *Concurrent Engineering: Research and Applications*, 1, No 1, pp 3—20 (1993).
- 11 Aoyama M: 'Concurrent development process model', *IEEE Software*, 10, No 4, pp 46—55 (July 1993).
- 12 Dedene G and De Vreese J P: 'Realities of off-shore reengineering', *IEEE Software*, 12, No 1, pp 35—45 (January 1995).
- 13 Gorton I and Motwani S: 'Issues in co-operative software engineering using globally distributed teams', *Information and Software Technology Journal* (August 1996)
- 14 Jones T C: 'Programming productivity', McGraw-Hill, New York (1986).
- 15 DeMarco T and Lister T: 'Peopleware', Dorset House, New York (1987).
- 16 Kraut R E and Streeter L A: 'Coordination in software development', *Comms ACM*, 38, No 3, pp 69—81 (March 1995).
- 17 Couloris G, Dollimore J and Kindberg T: 'Distributed systems concepts and design', 2nd Edition, Addison-Wesley (1994).
- 18 Burke K, Chidambaram L and Locke J: 'Evolution of relational factors over time — a study of distributed and non-distributed meetings', in Proceedings of the 28th Annual Hawaii International Conference on System Sciences, 4, Information Systems Track, Maui, pp 14—23, (January 1996).
- 19 Kawalek P: 'Coordinating people and technology through process', in Proceedings Information Technology and People, Moscow (1993).
- 20 Warboys B: 'The IPSE 2.5 project — a process model based architecture', in Bennet K (Ed): 'Software Engineering Environments: Research and Practice', Wiley, New York (1990).
- 21 Ben-Shaul I Z and Kaiser G E: 'A paradigm for decentralized process modeling and its realization in the Oz environment', in Proceedings of 16th International Conference on Software Engineering, Sorrento, Italy (May 1994).
- 22 Ben-Shaul I Z and Kaiser G E: 'A configuration process for a distributed software development environment', in 2nd International Workshop on Configurable Distributed Systems, Pittsburgh PA, pp 123—134 (March 1994).
- 23 Kitchenham B, Pickard L and Lawrence S P: 'Case studies for method and tool evaluation', *IEEE Software*, 12, No 3, pp 52—62 (July 1995).



CSIRO to join Transarc Corporation as a consultant, based in Sydney.

Ian Gorton received a BSc(Hons) and PhD in Computer Science from Sheffield Hallam University in 1985 and 1989 respectively. Since moving to Australia in 1989, he has gained both commercial and research experience in designing and building distributed systems in the areas of groupware and distributed multimedia information systems. His major research interests are software engineering methods for distributed systems, and process-centred software engineering support environments. At CSIRO he held the position of Research Leader in the Software Engineering Research Group. He has recently left



Prior to joining the University of Technology, Sydney, he has been a Principal Lecturer in Information Systems at the University of Canberra, a visiting Professor at the University of Maryland, College Park, and a Research Engineer at Telecom Australia. He received a PhD in Computer Science from MIT and Bachelor and Masters degrees in Electrical Engineering from the University of Adelaide.

Igor Hawryszkiewicz is Professor of Computing Sciences at the University of Technology, Sydney. Since 1988, his work has been in Computer Supported Co-operative Work (CSCW) concentrating on the design of networked enterprises. He has worked in this field on a number of projects with BT, the German National Computer Research Center (GMD), and the Research Institute of Applied Knowledge Processing (FAW) at the University of Ulm in Germany. His earlier research has been in systems development and design methodologies and he has published three text books in this area.



In addition to being the project manager for GWSE, he has been involved in the development and support of business applications used by the BT operational units in the region.

Kenny Ragoonaden received a BSc(Hons) degree in Physics in 1981 and an MSc Degree in Solid State Electronics in 1993, both from UMIST. In 1983 he joined BT, working in the data processing technical support unit. In 1991 he obtained a further MSc degree in Computer Science from City University, London. He moved to BT Laboratories in 1989, joining the software technology division where his work included investigating new operating system developments and the introduction of DCE (Distributed Computing Environment) to business applications. Since 1995, he has been working